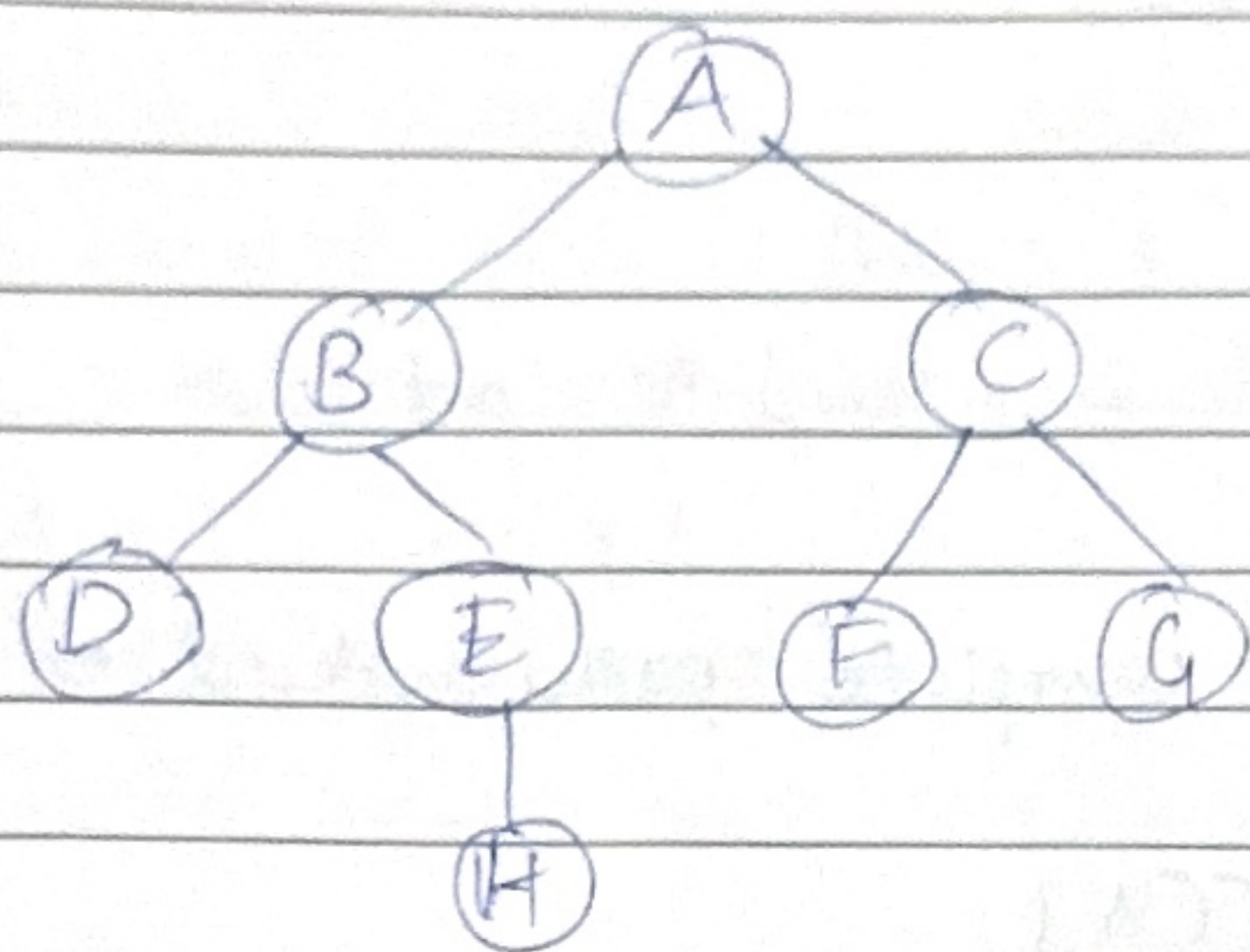


Lab-2a Breadth-First Search

Find the shortest path from the start node (A) to a goal node (H) in a graph using BFS algorithm



Step 1: Define the Graph

```
graph = {'A': ['B', 'C'],  
        'B': ['D', 'E'],  
        'C': ['F', 'G'],  
        'D': [],  
        'E': ['H'],  
        'F': [],  
        'G': [],  
        'H': []}
```

The graph is declared as a dictionary where the nodes are represented as keys, and child nodes as values.

Here:

A has children B and C

B has children D and E

C has children F and G

E has child H

Step 2: Define BFS Function

```
def bfs(start, goal):
```

This function performs Breadth-First Search to find a path from the start node to the goal node.

start $\rightarrow$ starting node

goal $\rightarrow$ destination node

Step 3: Initialize Queue and Visited List

```
queue = [start]
```

The queue stores complete paths instead of individual nodes.

```
Initially, queue = ['A']
```

```
visited = []
```

Stores the nodes that have already been explored.

Step 4: Start BFS Traversal

while queue:

The loop continues until there are no more paths left to explore.

Step 5: Remove path from queue

```
path = queue.pop(0)
```

Removes the first path from the queue.

pop(0) follows the FIFO principle (First in First Out)

$\hookrightarrow$ Main property of BFS

Ex: Queue = [A, B, C]

Remove A first

Step 6: Get Current node

```
node = path[-1]
```

The last element of the path represents the current node being explored.

Ex: path = ['A', 'B', 'E']

node = E

Step 7: Check goal and mark visited

if node == goal:

return path, visited

If the current node is the goal node, the path and visited order are returned.

if node not in visited:

visited.append(node)

If the node has not been visited, it is added to the visited list.

Step 8: Add child nodes to queue

for n in graph[node]:

queue.append(path + [n])

All child nodes of the current node are added to the queue along with the existing path.

Ex: A → B, C

The queue becomes:

['A', 'B']

['A', 'C']

Step 9: Execute BFS and display result

p, v = bfs('A', 'H')

calls BFS with:

Start node = A

Goal node = H

print("BFS path:", p)

print("BFS visited:", v)

Displays:

The shortest path from A to H.

The order in which nodes were visited.

Output:

BFS path: ['A', 'B', 'E', 'H']

BFS visited: ['A', 'B', 'C', 'D', 'E', 'F', 'G'] ~~['H']~~

Difference between BFS and DFS:-

BFS: path = queue.pop(0)

DFS: path = ~~stack~~.pop()

BFS uses queue (FIFO) i.e., it explores nodes level by level

DFS uses stack (LIFO) i.e., it explores one branch

completely before it moves to the next branch.